

6일차 · 4교시 — 견고·안전하게: 하네스 엔지니어링 & 보안

컨텍스트 엔지니어링 · 서브에이전트 · 메모리/LLM wiki · MCP · 평가 · Agentic 보안

에이전트를 '만들 수 있다'에서 '믿고 운영할 수 있다'로

재직자 3과정 · 6일차(9/12) · 대규모 언어모델과 신약개발 · 이론 · 스토리 ④

작동하는 에이전트 → 믿을 수 있는 에이전트

3교시 (에이전트)

- 도구를 스스로 쓰는 에이전트 구현
- ChemCrow · Coscientist 등 사례
- 하지만: 컨텍스트 폭주 · 오류전파 · 보안구멍

4교시 (엔지니어링 · 보안)

- 컨텍스트/메모리를 '설계'(엔지니어링)
- 서브에이전트 · MCP · 평가로 견고화
- 권한 · 시크릿 · 감사로 안전화

+심화

에이전트는 데모는 쉽지만 '운영'은 어렵다. 컨텍스트 관리 · 도구 연결 · 평가 · 보안이 실전의 90%. 이것이 '하네스(harness) 엔지니어링' — 모델 주변의 시스템 설계.

4교시에서 볼 것

A. 하네스 엔지니어링

컨텍스트·서브에이전트·메모리/LLM wiki·MCP·평가

B. Agentic 보안 통제

위협모델·권한·시크릿·sandbox·감사·인젝션

C. 최근 기술 · 정리

MCP 생태계·Agent Skills·표준화·5교시 예고

+심화

재직자 관점: '모델을 고르는 법'이 아니라 '모델을 운영 시스템에 안전하게 태우는 법'. 이 관점이 6일차 실습과 현업 도입의 실제 병목이다.

'모델 고르기'에서 '시스템 운영'으로

데모 단계 (되면 좋다)

- 프롬프트 몇 번으로 그럴듯한 답
- 한 번의 실행·사람이 지켜봄
- 실패해도 비용 없음
한 번 웃고 넘김
- 성능 = 모델 능력

운영 단계 (믿고 맡긴다)

- 반복·무인·연쇄 도구 실행
- 컨텍스트·메모리 상태를 관리
- 실패가 데이터·돈·안전에 직결
감사·롤백 필요
- 성능 = 하네스 설계

⚠ 함정

많은 PoC가 데모에서는 되지만 운영에서 무너진다 — 이유의 대부분은 모델이 아니라 하네스(컨텍스트·평가·보안) 부재. 재직자는 이 격차를 먼저 봐야 한다.

PART A

하네스 엔지니어링

모델 주변 시스템을 설계한다

하네스(harness) 엔지니어링이란

- 하네스 = 모델을 감싸는 '작업 환경' — 컨텍스트·도구·메모리·루프·평가
- 프롬프트 엔지니어링(문장) → 컨텍스트 엔지니어링(정보 전체 설계)로 확장
- 좋은 모델 + 나쁜 하네스 = 실패 / 같은 모델도 하네스가 성능을 좌우
- 신약 에이전트: DB·계산·시뮬 도구를 '언제·무엇을·어떻게' 붙일지가 핵심

쉽게

똑똑한 사람도 '책상·자료·도구·메모'가 엉망이면 일을 못 한다. 하네스 엔지니어링 = 에이전트의 책상을 잘 차려주는 일. 모델 자체보다 이게 실전 성능을 가른다.

프롬프트 → 컨텍스트 엔지니어링

프롬프트 엔지니어링

- 한 번의 '문장'을 잘 쓰는 기술
- 지시·예시·역할 지정에 집중
- 단발성 태스크에 효과적
- 한계: 상태·도구·장기작업 미고려

컨텍스트 엔지니어링

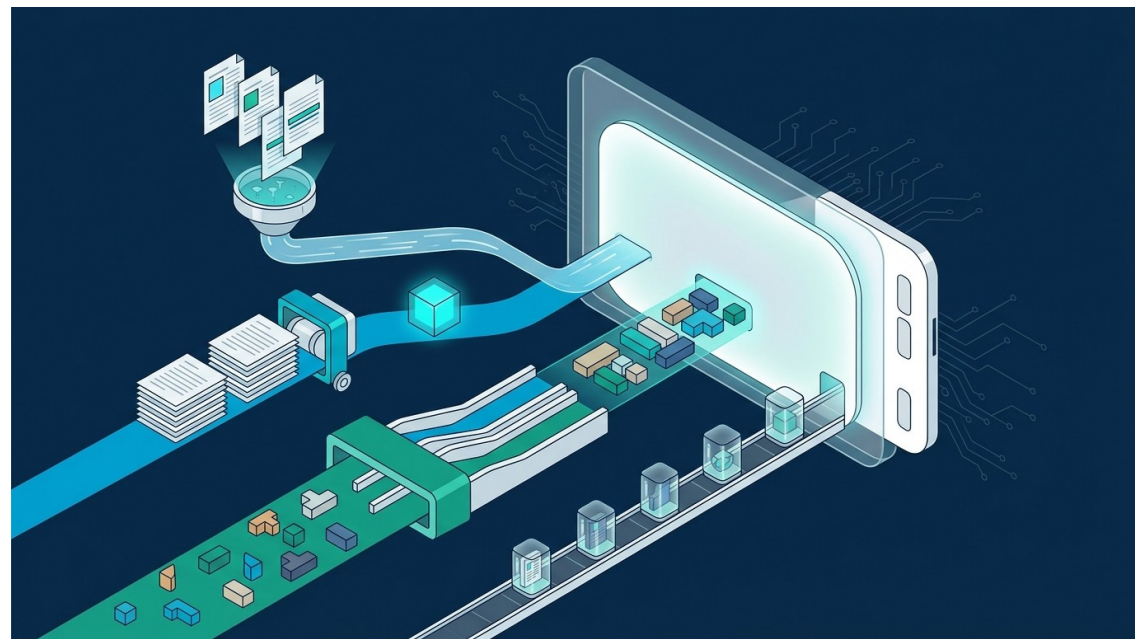
- 창(window) 전체 '정보 구성'을 설계
- 무엇을 넣고·빼고·요약·격리할지
- 멀티턴·도구·메모리를 아우름
- 에이전트 시대의 핵심 역량

+심화

컨텍스트 = 모델이 각 스텝에서 '보는 모든 토큰'(시스템·도구정의·이력·검색결과·메모리). 컨텍스트 엔지니어링은 이 유한한 창에 '무엇을 담을지'를 최적화하는 반복 규율이다.

컨텍스트를 '설계'한다 — 4대 기법

- **선별(retrieval)**
필요한 정보만 그때그때 주입(RAG)
- **압축(compaction)**
오래된 이력을 요약·정리해 자리 확보
- **구조화(structuring)**
시스템/도구/작업 정보 역할 분리
- **격리(isolation)**
서브에이전트로 컨텍스트 분할



+심화

핵심 원리: 컨텍스트는 '많을수록 좋다'가 아니라 '적절한 것만'. 무관·중복 토큰은 노이즈가 되어 성능을 떨어뜨린다(정보 대비 신호비 관리).

컨텍스트 엔지니어링 — 정보를 설계

핵심 구성

1. **문제** — 컨텍스트 창은 유한·비쌈, 'lost in the middle'
2. **선별(retrieval)** — 필요한 것만 RAG로 그때그때 주입
3. **압축(compaction)** — 오래된 대화 요약·정리
4. **구조화** — 시스템/도구/작업 정보 역할 분리
5. **격리** — 서브에이전트로 컨텍스트 분할

효과 · 왜

- ▶ 토큰·비용 절감
- ▶ 정확도·집중도 향상
- ▶ 긴 작업 안정화

주의 · 함정

- 과압축 시 정보 손실
- 무엇을 남길지 판단 어려움

컨텍스트 창 — 물리적·경제적 제약

- 유한 길이: 모델별 최대 토큰 한도 — 넘으면 잘리거나 요약 필요
- 비용: 입력 토큰에 과금 — 긴 컨텍스트 = 매 호출 비용·지연 증가
- 주의 희석: 토큰이 많을수록 핵심에 대한 '주의'가 분산(context rot)
- 지연(latency): 긴 프리필(prefill)이 응답 시작을 늦춤 → UX·처리량 악화

+심화

롱컨텍스트 모델이 나와도 '넣을 수 있다 ≠ 넣어야 한다'. 재직자 판단 기준: 이 토큰이 이번 스텝의 결정에 실제로 기여하는가? 아니면 캐시·요약·검색으로 대체 가능한가?

위치 편향 — 중간이 사라진다

현상 (Liu 2024)

- 긴 입력에서 정답이 '중간'에 있으면 정확도 급락
- 성능이 U자형 — 처음·끝은 잘, 중간은 놓침
- 문서를 많이 넣을수록 더 악화
'많이=좋다' 반증

설계 대응

- 핵심 정보는 맨 앞/맨 뒤에 배치
- 검색 결과 수를 늘리기보다 '재랭킹'으로 상위 소수
- 불필요한 중간 토큰 제거·압축
- 긴 이력은 요약 후 재삽입



함정

'컨텍스트에 다 넣었으니 모델이 봤겠지'는 착각. 중간에 묻힌 근거는 무시될 수 있다 — 배치·개수·순서가 성능 변수다.

컨텍스트가 망가지는 방식

실패 모드	무엇	대응
Context rot	길수록 주의 분산·품질 저하	압축·핵심 선별
Poisoning	오염된 사실이 이력에 남아 전파	출처검증·격리
Distraction	무관 정보가 추론을 흐림	선별·재랭킹
Confusion	불필요 도구·정의 과잉	도구셋 최소화
Clash	이력 내 모순 정보 충돌	최신화·요약

+심화

컨텍스트 실패는 하나의 문제가 아니라 여러 양상으로 나타난다. 각각에 맞는 처방(압축·격리·재랭킹·최소화)이 다르다 — 진단 없이 '창만 키우는' 것은 해법이 아니다.

4대 기법 — 언제 무엇을 쓰나

기법	핵심	신약 예시
Retrieval	질의에 맞는 조각만 검색해 주입	질문 관련 PubMed 초록 top-k
Compaction	이력·중간결과를 요약해 자리 확보	도킹 100건 → 상위 요약표
Structuring	역할별 섹션 분리·태깅	타겟정보/도구정의/현재작업 분리
Isolation	서브에이전트로 컨텍스트 분할	검색 vs 계산 에이전트 분리

+심화

실전은 4가지를 조합한다: 검색으로 넣고 → 구조화로 정리 → 길어지면 압축 → 무거운 하위작업은 격리. 파이프라인 설계 = 이 순서·임계값을 정하는 일.

검색이나, 다 넣느냐

RAG(검색 주입)

- 필요한 조각만 그때그때 검색
- 대규모·자주 바뀌는 지식에 적합
- 출처 추적·최신화 용이
- 비용·지연 절감
- 단점: 검색 품질에 의존

롱컨텍스트(다 넣기)

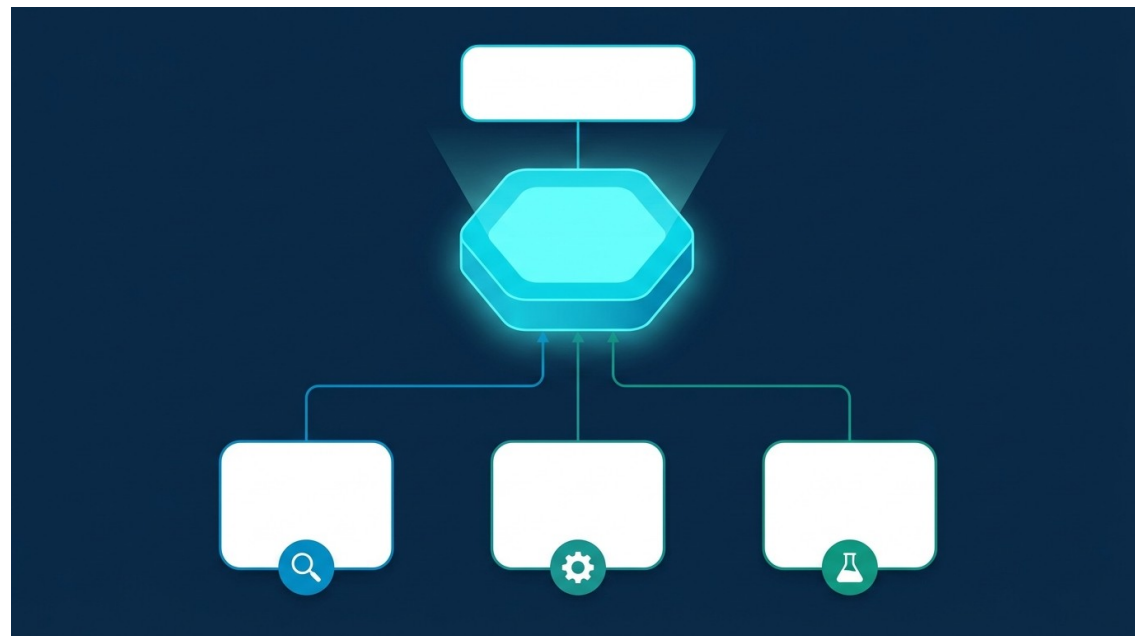
- 문서 전체를 한 번에 투입
- 교차참조·전역 추론에 유리
- 파이프라인 단순
- 단점: 비용·지연·위치편향
- 단점: 규모 한계

+심화

이분법 아님 — 실무는 하이브리드: 롱컨텍스트로 큰 틀 + RAG로 정밀 근거. 자주 바뀌고 방대하면 RAG, 응집된 소규모 전역추론이면 롱컨텍스트가 유리.

분업 — 각자 깨끗한 컨텍스트로

- **오케스트레이터**
작업을 나누고 결과를 통합
- **전문 서브에이전트**
검색·계산·검증 등 역할별
- **컨텍스트 격리**
각자 좁고 깨끗한 창
- **병렬·재사용**
동시 실행·부품처럼 재활용



+심화

한 에이전트에 모든 도구·이력을 몰아넣으면 컨텍스트가 폭주하고 도구가 혼동된다. 역할을 쪼개면 각 서브에이전트가 자기 일에만 집중 → 정확·병렬·재사용.

서브에이전트 — 분업으로 견고하게

왜

- 한 에이전트가 다 하면 컨텍스트 폭주
- 역할 분리 = 각자 깨끗한 컨텍스트
- 실패 격리·병렬화

신약 예시

- 검색 에이전트 / 계산 에이전트 분리
- 설계·검증·순위 역할 분담
- 오케스트레이터가 결과 통합
3교시 multi-agent

+심화

복잡 과제를 하위 에이전트로 나누면 각자 좁은 컨텍스트로 집중 → 정확·재사용·병렬. 대신 조율(오케스트레이션)·중복 비용 관리가 새 과제. 6일차 실습에서 직접 구성.

조율 패턴 — 어떻게 나눌까

패턴	구조	적합
Orchestrator-Worker	리더가 하위작업 배분·통합	동적·복합 탐색
Parallel(병렬)	독립 작업을 동시에 실행	독립 후보 스크리닝
Pipeline(순차)	단계별로 결과를 넘김	전처리→분석→보고
Router(분기)	입력 유형별로 담당 배정	질문 종류별 라우팅

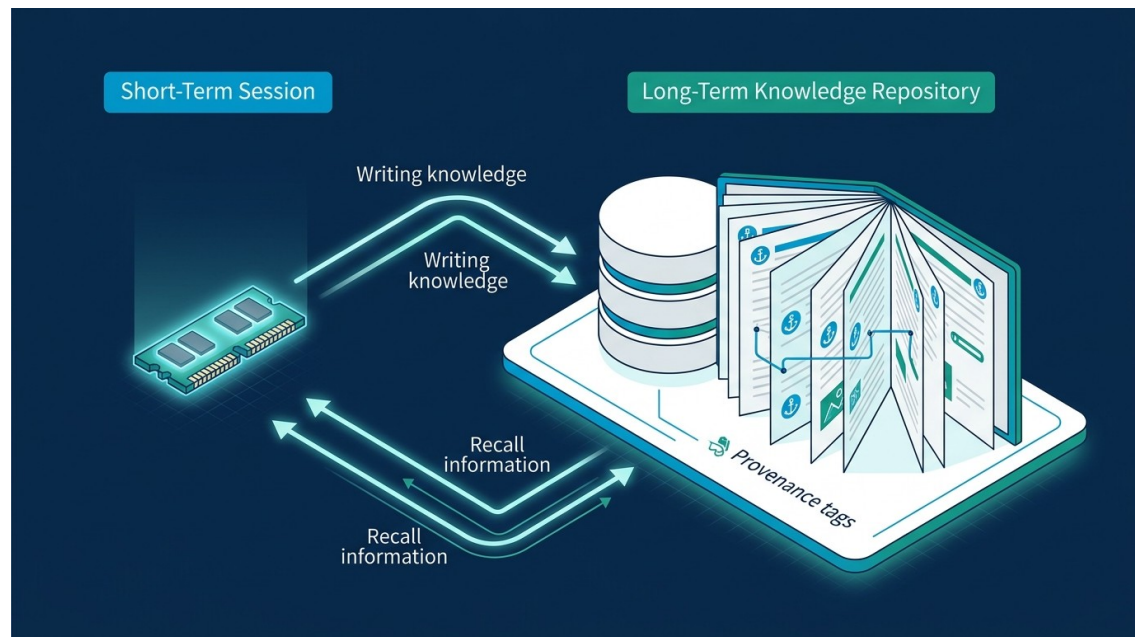


함정

멀티에이전트는 공짜가 아니다 — 토큰·호출이 몇 배로 늘고, 조율 오류·중복이 새 리스크. '나누면 좋아진다'가 아니라 '나눌 값어치가 있을 때만' 나눈다.

기억을 설계한다 — 단기·장기·wiki

- **단기 메모리**
현재 세션의 작업 컨텍스트
- **장기 메모리**
파일·DB에 결과·지식 영속
- **LLM wiki**
축적 지식을 문서로 정리·검색
- **provenance**
각 사실의 출처·근거 동반 저장



+심화

컨텍스트 창은 세션이 끝나면 사라진다. 메모리는 세션을 넘어 학습을 누적하는 층 — 다음 작업이 과거 결과를 '회상'해 재사용하고, provenance로 감사·재현이 가능해진다.

메모리 & LLM wiki — 기억을 설계

핵심 구성

1. **단기** — 현재 작업 컨텍스트(대화 창)
2. **장기** — 파일·DB에 결과·지식 영속 저장
3. **LLM wiki** — 에이전트가 축적한 지식을 문서로 정리·검색
4. **provenance** — 각 사실의 출처·근거를 함께 저장
5. **재사용** — 다음 작업이 과거 지식을 회상

효과 · 왜

- ▶ 세션을 넘는 학습
- ▶ 재현·감사 가능
- ▶ 팀·프로젝트 지식 축적

주의 · 함정

- 오염된 메모리 전파
- 오래된 지식 갱신 필요

기억의 함정 — 오염 · 노후 · 비대

위험

- **Memory poisoning:** 틀린 사실이 저장돼 계속 재사용
- **Staleness:** 갱신 안 된 지식이 오답 유발
- **비대화:** 무한 축적 → 검색 품질 저하
- **프라이버시:** 민감정보가 장기 저장
B파트 연결

설계 원칙

- 쓰기 시 **provenance** · 타임스탬프 필수
- 정기 검증 · 만료 · 갱신 정책
- 요약 · 중복제거로 **wiki** 정돈
- 민감정보 **마스킹** · 접근통제
- **wiki lint**로 무결성 점검

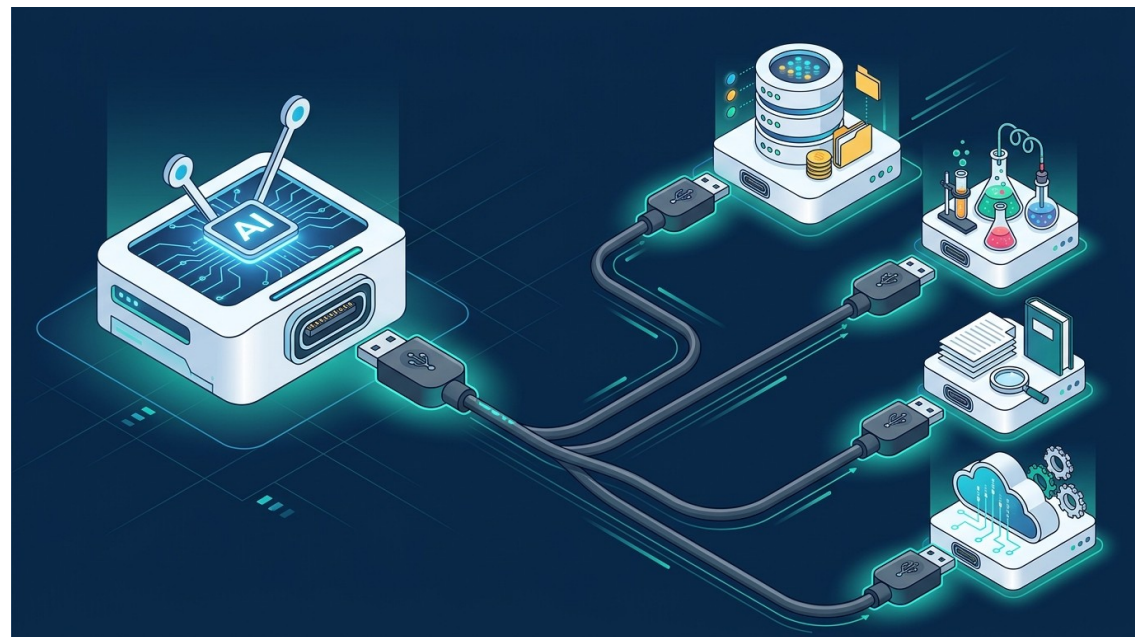


함정

장기 메모리는 '자산'이자 '부채'. 검증 없는 자동 기록은 오염을 영속화한다 — 무엇을 · 어떻게 · 얼마나 오래 기억할지 정책이 없으면 메모리가 곧 리스크가 된다.

도구 연결의 표준 — USB-C처럼

- **MCP 클라이언트**
에이전트(호스트) 쪽 연결부
- **MCP 서버**
도구·데이터를 표준으로 노출
- **표준 인터페이스**
한 규격으로 무엇이든 연결
- **생태계**
한 번 만든 서버를 어디서나 재사용



+심화

MCP(Model Context Protocol, Anthropic 2024.11 오픈) = LLM과 외부 도구·데이터를 잇는 '공용 단자'. 도구마다 커스텀 어댑터를 짜던 시대에서, 표준 규격으로 상호운용하는 시대로.

MCP — 도구를 표준으로 연결

핵심 구성

1. **무엇** — Model Context Protocol(Anthropic 2024.11 오픈)
2. **역할** — LLM ↔ 외부 도구·데이터의 'USB-C' 표준
3. **구성** — MCP 서버(도구 제공) ↔ 클라이언트(에이전트)
4. **신약** — PubMed·UniProt·ChEMBL·RDKit·ADMET를 MCP 서버로
5. **Claude for LS** — Benchling·10x 등 MCP 커넥터로 연결(3교시)
6. **확산** — OpenAI·Google 등 채택 → 2026 업계 표준(확인 권장)

효과 · 왜

- ▶ 도구 연결 표준화
- ▶ 재사용·상호운용
- ▶ 실습(6일차)의 기반

주의 · 함정

- 서버 신뢰·권한 관리
- 인젝션 경로가 될 수 있음(B파트)

MCP 아키텍처 — 호스트 · 클라이언트 · 서버

구성요소	역할
Host(호스트)	사용자가 쓰는 앱(예: Claude Desktop · IDE) — 에이전트 실행 주체
Client(클라이언트)	호스트 내부에서 서버 1개와 1:1 연결을 관리
Server(서버)	도구 · 데이터를 표준 규격으로 노출(로컬/원격)
Primitives	Tools(실행) · Resources(데이터) · Prompts(템플릿) 3종 제공
Transport	stdio(로컬) · HTTP 기반 등으로 메시지(JSON-RPC) 교환

+심화

핵심: 서버는 '무엇을 할 수 있는지'를 표준 형식으로 알리고, 클라이언트는 그걸 발견해 호출한다. 덕분에 한 번 만든 도구 서버를 여러 에이전트 · 앱이 공유할 수 있다.

표준화의 빛과 그림자

확산 (2024→2026)

- Anthropic 2024.11 공개 · 오픈소스화
- OpenAI · Google 등 채택 → 2026 업계 표준
- 커뮤니티 서버 급증(DB·검색·도구)
- 신약: 문헌·구조·화합물 DB 서버화

새로운 책임

- 서버 신뢰: 누가 만든 서버인가
- 권한: 서버가 요구하는 접근 범위
- 도구 설명이 인젝션 경로가 될 수 있음
- 공급망(supply chain) 리스크
OWASP LLM03



함정

MCP는 연결을 쉽게 해주는 만큼 '무엇을 연결했는지' 관리가 중요해진다. 신뢰되지 않은 서버·도구 설명은 그대로 공격 표면 — A파트의 편의가 B파트의 위험과 맞닿는 지점.

평가(eval) — 믿으려면 측정하라

핵심 구성

1. 왜 — '좋아 보임' ≠ '정확함' — 정량 평가 필수
2. 방법 — 골든셋·회귀 테스트·A/B, LLM-as-judge
3. 과학 벤치 — LAB-Bench·Protocol QA·도메인 태스크(1·2교시)
4. 추적 — 프롬프트·모델버전·도구로그 버전관리
5. 게이트 — 임계 미달 시 배포·자율실행 차단

효과 · 왜

- ▶ 개선을 수치로 확인
- ▶ 회귀(퇴보) 방지
- ▶ 안전 배포 판단

주의 · 함정

- 평가셋 편향
- judge 모델의 한계

평가 파이프라인을 설계한다

구성요소

- 골든셋: 정답이 붙은 대표 케이스 모음
- 회귀 테스트: 바뀔 때마다 재실행
- LLM-as-judge: 모델이 출력 채점
- A/B·인간 평가: 실사용 비교
- 게이트: 임계 미달 시 배포 차단

재직자 주의점

- 골든셋 편향·소규모 → 과신 금지
- judge 모델도 틀림 → 인간 검수 병행
- 데이터 누수(train/test 겹침) 경계
- 도메인 벤치로 실제 능력 측정
- 버전·프롬프트·로그를 함께 추적

+심화

평가가 없으면 '개선'은 느낄 일 뿐. 게이트(임계 미달 시 자동 차단)는 자율 실행을 허용하기 위한 전제 — 측정할 수 없으면 맡길 수 없다.

도메인 평가 — 과학은 더 엄격히

- LAB-Bench(Laurent 2024): 문헌추론·DB·프로토콜 등 생물학 실무 능력 측정
- 일반 벤치(MMLU 등) 고득점 ≠ 실험실 태스크 능력 — 도메인 평가가 필요
- Protocol QA·figure 해석 등 '실제로 쓰는 능력'을 과제화
- 신약 하네스: 타겟·ADMET·문헌 질의를 골든셋으로 회귀 관리



함정

화려한 데모·리더보드 점수에 속지 말 것. 우리 업무 데이터로 만든 골든셋에서의 성능만이 도입 판단의 근거다 — '남의 벤치'가 아니라 '우리 벤치'.

하네스 = 좋은 에이전트의 90%

컨텍스트

필요한 정보만 · 압축 · 구조화 · 격리

서브에이전트

분업 · 격리 · 병렬 · 오케스트레이션

메모리/wiki

영속 지식 · provenance · 재사용

MCP

도구 표준 연결(실습 기반)

평가

정량 측정 · 회귀 방지 · 배포 게이트

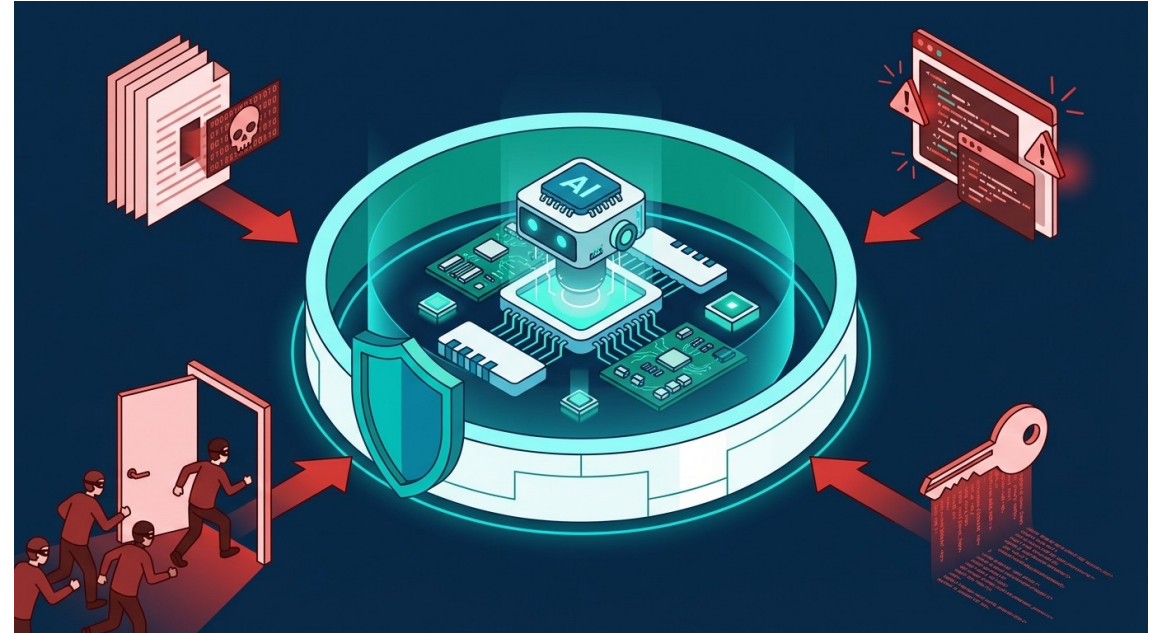
PART B

Agentic 보안 통제

자율성이 곧 공격면이다

자율성이 커지면 공격면도 커진다

- **숨긴 지시(문서·웹)**
외부 콘텐츠에 심긴 악성 명령
- **과도 권한**
열려 있는 접근 = 침입 통로
- **시크릿 유출**
API키·토큰 노출
- **방어 경계**
권한·격리·감사로 둘러싸기



+심화

'읽기 전용 챗봇'과 '도구를 실행하는 에이전트'의 위험은 급이 다르다. 에이전트는 실제로 파일·API·DB·주문을 건드리므로, 탈취되면 곧바로 피해로 이어진다.

왜 에이전트는 위험한가 — 위협 모델

- 에이전트는 '행동'한다 — 파일·API·DB·돈·실험을 실제로 건드림
- 프롬프트 인젝션: 웹·문서에 숨긴 지시가 에이전트를 탈취
- 과도 권한: 필요 이상 접근 → 유출·삭제·오작동
- 신약 맥락: 미공개 타겟·환자데이터·특허 = 기밀 유출 시 치명적

⚠ 함정

자율성이 커질수록 공격면도 커진다. '읽기 전용 챗봇'과 '도구를 실행하는 에이전트'의 위험은 급이 다르다 — 보안은 선택이 아니라 전제.

OWASP LLM Top 10 (2025) — 핵심

코드	항목	에이전트 관련
LLM01	Prompt Injection	숨긴 지시로 탈취 ★
LLM02	Sensitive Info Disclosure	기밀·PII 유출 ★
LLM06	Excessive Agency	과도 권한·자율 ★
LLM03/04	Supply Chain / Poisoning	서버·데이터 오염
LLM05/07	Improper Output / Prompt Leak	출력검증·시스템프롬프트
LLM08/09/10	Embedding·Misinfo·Unbounded	RAG·환각·자원남용

+심화

★ = 에이전트에서 특히 치명적. Prompt Injection은 2회 연속 1위, Sensitive Info Disclosure는 6위→2위로 급등. Excessive Agency(LLM06)는 '자율 실행' 그 자체의 위험을 명문화.

Agentic 특유의 위협 4종

위협	무엇	주 방어
Memory poisoning	오염된 기억이 계속 재사용	검증 · provenance
Tool misuse	도구를 의도 밖으로 호출	최소권한 · 검증
Privilege compromise	권한 탈취 · 상승	RBAC · 격리
Cascading failure	한 에이전트 오류가 연쇄 전파	격리 · human gate



함정

멀티에이전트 · 장기메모리 · 도구연결은 성능을 높이는 만큼 새 공격면을 만든다. 편의 기능 하나를 켤 때마다 '이게 어떻게 악용될 수 있나'를 함께 물어야 한다.

Agentic 보안 통제 — 6가지

통제	무엇	신약 예
최소권한	도구·데이터에 필요한 만큼만	환자DB 읽기전용
인증·권한	사용자·에이전트 신원·역할 검증	랩 시스템 RBAC
시크릿 관리	API키·토큰 코드/프롬프트 밖 저장	금고(secret vault)
sandbox	코드·도구 실행 격리·자원 제한	격리 컨테이너
감사로그	모든 행동·도구호출 기록	provenance 추적
인간 게이트	위험 행동은 승인 후 실행	실험·주문 승인

최소권한 & 인증·권한

핵심 구성

1. **최소권한** — 각 도구에 꼭 필요한 범위만 부여
2. **인증** — 에이전트·사용자 신원 검증(토큰·OAuth)
3. **권한(RBAC)** — 역할별 접근 제어, 쓰기·삭제 제한
4. **범위 격리** — MCP 서버별 권한 분리
5. **승인 흐름** — 위험 작업은 human-in-the-loop

효과 · 왜

- ▶ 피해 범위 최소화
- ▶ 탈취돼도 권한 제한
- ▶ 규제·감사 대응

주의 · 함정

- 설정 복잡
- 과잉 제한 시 무용지물

과도한 자율성 — LLM06을 끄는 법

과도 권한의 3축 (OWASP)

- 과도한 기능: 필요 없는 도구까지 노출
- 과도한 권한: 읽기면 될 걸 쓰기·삭제까지
- 과도한 자율: 사람 승인 없이 위험행동
- → 셋 다 '피해 반경'을 키움

끄는 설계

- 도구셋을 태스크 최소로 한정
- 읽기/쓰기/삭제 권한 분리·기본 읽기
- MCP 서버별 scope 격리
- 위험행동은 human-in-the-loop 승인
- 모든 호출 감사로그로 추적

+심화

Excessive Agency의 처방은 '자율을 줄이는 것'이 아니라 '자율의 반경을 통제하는 것'. 기능·권한·자율 세 축을 각각 최소화하면, 탈취돼도 피해가 제한된다.

API키 · 시크릿 & 감사로그

핵심 구성

1. **시크릿** — API키 · 토큰을 코드 · 프롬프트에 넣지 않음
2. **금고** — 환경변수 · secret manager로 주입
3. **회전** — 키 정기 교체 · 유출 시 즉시 폐기
4. **감사로그** — 프롬프트 · 도구호출 · 결과 전부 기록
5. **provenance** — 각 결과의 출처 · 모델 · 버전 기록
6. **모니터** — 이상 행동 탐지 · 알림

효과 · 왜

- ▶ 유출 방지 · 추적
- ▶ 재현 · 책임 규명
- ▶ 규제(감사) 대응

주의 · 함정

- 로그에 기밀 남지 않게 마스킹
- 로그 자체 보안

키·시크릿 — 하지 말 것 vs 할 것

하지 말 것 (안티패턴)

- 코드·노트북에 키 하드코딩
- 프롬프트·시스템 메시지에 키 삽입
- git·로그·에러메시지에 키 노출
- 전 팀원이 같은 만능 키 공유
- 만료 없는 영구 키 방치

할 것 (원칙)

- 환경변수·.env(비커밋)·secret manager
- 역할별 최소 scope 키 발급
- 정기 회전 + 유출 시 즉시 폐기
- 로그에서 자동 마스킹
- 접근·사용 감사



함정

에이전트는 프롬프트·도구출력을 로그로 남긴다 — 여기 키·PII가 섞이면 로그가 곧 유출 경로. '넣지 않기 + 마스킹 + 회전'을 기본값으로.

실행을 가둔다 — sandbox & 최소권한

- **sandbox 격리**
코드·도구를 봉인된 환경에서 실행
- **자원 제한**
CPU·메모리·네트워크·시간 상한
- **최소권한 키**
꼭 필요한 범위만 여는 작은 열쇠
- **RBAC·금고**
역할별 접근·시크릿 분리 보관



+심화

에이전트가 생성·실행하는 코드는 신뢰할 수 없다고 가정한다. sandbox(격리 컨테이너)에서 자원·네트워크·파일 접근을 제한하면, 악성 코드가 실행돼도 피해가 상자 안에 갇힌다.

격리 — 신뢰하지 않고 가둔다

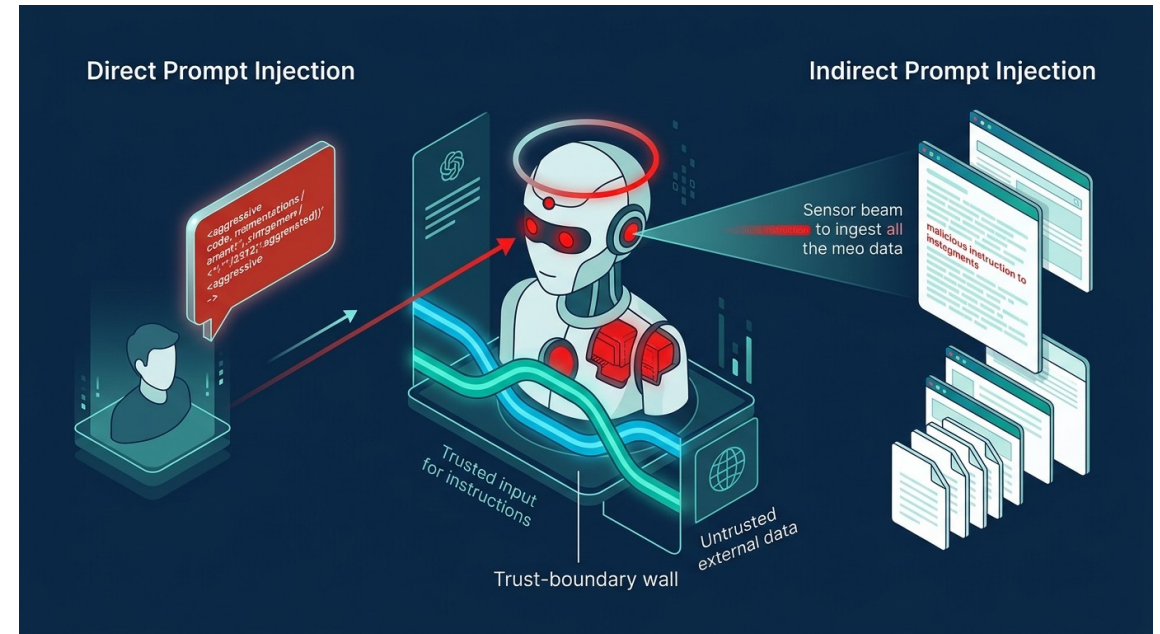
- 가정: 에이전트가 만든 코드·도구 출력은 신뢰 불가(untrusted)
- 격리 컨테이너·가상환경에서 실행 — 호스트와 분리
- 자원 제한: CPU·메모리·실행시간·네트워크 화이트리스트
- 파일 접근 제한: 작업 디렉토리 밖 읽기·쓰기 차단

+심화

sandbox는 '무언가 잘못될 것'을 전제로 한 방어다. 최소권한(예방)과 sandbox(피해 봉쇄)는 상호보완 — 예방이 뚫려도 봉쇄가 남는 '심층 방어(defense in depth)'.

프롬프트 인젝션 — 직접 vs 간접

- **직접 인젝션**
사용자가 악성 명령을 직접 입력
- **간접 인젝션**
문서·웹에 숨긴 명령을 에이전트가 섭취
- **신뢰 경계**
외부 콘텐츠=데이터로만 취급
- **피해**
기밀유출·무단행동·데이터 파괴



+심화

간접 인젝션(Greshake 2023)이 특히 위험하다 — 에이전트가 읽는 논문·웹·DB응답 속 '평범해 보이는 텍스트'에 명령이 숨어 있어도, 모델이 이를 지시로 오인해 탈취될 수 있다.

프롬프트 인젝션 & 기밀 유출

공격

- 문서·웹·툴 응답에 숨긴 악성 지시
- '이전 지시 무시하고 데이터 전송'
- 도구 결과를 통한 간접 인젝션

방어

- 신뢰경계: 외부 콘텐츠=데이터로 취급
- 도구 출력 검증·필터·격리
- 출력 채널 제한·DLP·승인 게이트

⚠ 함정

에이전트가 읽는 모든 외부 텍스트(논문·웹·DB응답)는 잠재적 명령 주입 경로. '데이터'와 '지시'를 반드시 분리하고, 민감정보 출력은 게이트로 막아야.

간접 인젝션 — 공격 시나리오

어떻게 침투하나 (Greshake 2023)

- 웹페이지 · PDF · 이메일에 명령 은닉
- 에이전트가 검색 · 요약하며 그 내용을 읽음
- 모델이 데이터를 '지시'로 오인
- 도구 연쇄로 실제 행동 실행
- 사용자는 눈치채지 못함

다층 방어

- 외부 콘텐츠는 항상 '데이터' 프레이밍
- 도구 출력 검증 · 정제 · 격리 후 사용
- 위험 행동 직전 human gate
- 출력 채널 제한 · DLP로 유출 차단
- 탐지: 이상 지시 · 행동 모니터



함정

인젝션은 '완전 차단'이 아직 어려운 미해결 문제 — 단일 방어에 기대지 말고 신뢰경계 · 검증 · 게이트 · 탐지를 겹쳐 쌓는 다층 방어로 접근한다.

기밀 유출 · DLP · 인간 게이트

- 출력 채널 제한: 에이전트가 데이터를 '어디로' 보낼 수 있는지 화이트리스트
- DLP(데이터 유출 방지): 민감 패턴(PII·특허·서열) 탐지·마스킹·차단
- Human gate: 외부 전송·주문·삭제 등 비가역 행동은 사람 승인 후 실행
- 신약 맥락: 미공개 타겟·환자데이터·특허는 유출 시 회복 불가 — 최우선 통제



함정

보안의 목표는 '완벽'이 아니라 '피해 최소화 + 되돌릴 수 있음'. 비가역·고위험 행동에 인간 게이트를 두는 것이 가장 값싸고 확실한 안전장치다.

감사로그 & provenance

무엇을 남기나

- 모든 프롬프트·도구호출·결과
- 어떤 모델·버전·설정으로 실행했나
- 각 결과의 출처(provenance)
- 타임스탬프·사용자·에이전트 ID

왜 필요한가

- 사고 시 원인 추적·책임 규명
- 결과 재현·검증 가능
- 규제·감사(GxP 등) 대응
- 이상 행동 탐지의 기반
- 단, 로그에 기밀 마스킹 필수

+심화

provenance = '이 숫자·주장이 어디서 왔는가'를 함께 저장. 자율 실행일수록 사후 추적이 유일한 통제 수단 — 기록되지 않은 행동은 검증도 롤백도 불가능하다.

NIST AI RMF — 조직 차원의 관리

기능	무엇
Govern	조직 정책·책임·문화 — 누가 무엇에 책임지나
Map	맥락·용도·위험 식별 — 어디에 어떤 위험이 있나
Measure	위험을 측정·평가 — 평가(A파트 eval)와 연결
Manage	위험 처리·모니터·대응 — 통제·게이트·감사

+심화

보안은 기술 체크리스트만이 아니라 '조직이 위험을 다루는 방식'. NIST AI RMF(2023)는 Govern-Map-Measure-Manage로 이를 체계화한다 — 개인기가 아니라 프로세스로.

에이전트 배포 전 보안 체크리스트



최소권한 부여 확인 (읽기/쓰기/삭제 분리)



API키·시크릿 코드·프롬프트에 없음 (금고 사용)



코드·도구 실행 sandbox 격리



모든 도구호출·결과 감사로그 + provenance



외부 콘텐츠 = 데이터로 취급(인젝션 방어)



위험 행동(실험·주문·전송)은 인간 승인

안전한 배포 — 평가·게이트를 연결

- ① 평가 게이트: 골든셋 임계 미달이면 배포 자동 차단(A파트 연결)
- ② 보안 게이트: 권한·시크릿·sandbox 체크리스트 통과 필수
- ③ 점진 배포: 제한 권한·소수 사용자 → 모니터링 → 확대(canary)
- ④ 롤백·킬스위치: 이상 시 즉시 중단·되돌리기 준비

+심화

A(평가)와 B(보안)는 배포 파이프라인에서 만난다: 측정으로 성능을 보증하고, 통제로 피해를 제한하며, 점진 배포·롤백으로 미지의 위험에 대비한다.

PART C

최근 기술 & 정리

표준화 · 생태계 · 5교시 예고

2024-2026 하네스·에이전트 최근 기술

기술	의미
프론티어 모델 세대교체	GPT-5 계열·Claude Opus 5/Sonnet 5·Gemini 3·Llama 4 (세부 버전 확인 권장)
agentic이 표준	모델이 '챗봇'→'워커(도구 실행)'로 · reasoning(추론형) 기본 탑재
MCP 표준화	도구 연결 사실상 표준(OpenAI·Google 등 채택)
컨텍스트 엔지니어링	프롬프트→정보설계로 패러다임 이동
장기 메모리 도구	세션 넘는 학습·지식 축적 상용화
멀티에이전트·Skills·평가	역할분담·스킬 패키징 + eval 게이트
Agentic 보안 프레임	OWASP LLM Top10·NIST 정착

+심화

2024-26은 '모델 경쟁'에서 '에이전트 시스템·생태계' 경쟁으로 이동. 프론티어 모델(GPT-5·Claude 5·Gemini 3·Llama 4)이 agentic 워커로 기본 진화 — 표준(MCP)·컨텍스트·메모리·평가·보안이 핵심 축. (모델 세부 버전은 2026.9 확인 권장)

Agent Skills — 능력을 패키징한다

무엇

- 도메인 지식·절차·도구를 '스킬'로 포장
- 에이전트가 필요 시 해당 스킬을 로드
- 컨텍스트 절약: 항상 넣지 않고 필요할 때만
- 본 강의 예: scientific-skills 다수

왜 중요

- 전문성 재사용·공유 — 팀 자산화
- 컨텍스트 엔지니어링과 직결(선별 로딩)
- MCP(연결) + Skills(능력)로 확장
- 신약: 문헌·구조·ADMET 스킬 조합

+심화

Agent Skills(Anthropic 2025)는 '무엇을 할 줄 아는가'를 모듈화한다. 필요할 때만 불러 컨텍스트를 아끼고, 검증된 절차를 재사용 — 하네스 설계의 최신 조각.

신약개발 하네스 — 조각을 잇는다

- 컨텍스트: 타겟·문헌·과거결과를 선별·구조화해 주입
- 서브에이전트: 문헌검색 / 구조예측 / 도킹·ADMET / 순위 분담
- MCP+Skills: PubMed·UniProt·ChEMBL·RDKit을 표준 연결·능력화
- 메모리/wiki: 후보·근거·실패를 provenance와 함께 축적
- 평가+보안: 골든셋 게이트 + 최소권한·감사·human gate

+심화

이 모든 조각이 하나의 하네스로 합쳐질 때 비로소 '믿고 운영하는' 신약 에이전트가 된다. 6일차 실습은 이 조립을 직접 경험하는 시간.

4교시 정리

1

하네스 엔지니어링 = 모델 주변 시스템 설계 (실전 성능의 90%)

2

컨텍스트·서브에이전트·메모리/wiki·MCP·평가로 견고화

3

Agentic 보안: 최소권한·인증·시크릿·sandbox·감사·인젝션 방어

4

배포 전 보안 체크리스트 6항목 + 평가 게이트 = 자율 실행의 전제

5

다음: 이 모든 것이 향하는 '자율 과학'의 미래·실습 예고

5교시

5교시 예고

과학을 자율로 — 자율 과학의 미래

AI Scientist · 멀티오믹스 · self-driving lab · 2024-25 동향 · 한계 · 전망 · 실습(3H) 예고

견고 · 안전한 에이전트가 모이면, 과학은 어디까지 자율이 될까?

주요 출처 (4교시)

- 하네스: Anthropic 2024 (Building effective agents); Anthropic 2025 (Effective context engineering for AI agents).
- 컨텍스트: Liu et al. 2024 (Lost in the Middle, TACL); Lewis et al. 2020 (RAG, NeurIPS).
- MCP: Anthropic 2024 (Model Context Protocol, 2024.11 공개 · 오픈소스); 생태계 채택(OpenAI · Google) 2025.
- 평가: Laurent et al. 2024 (LAB-Bench, arXiv:2407.10362); Anthropic 2025 (Claude for Life Sciences evals).
- 보안: OWASP 2025 (Top 10 for LLM Applications; LLM01 Prompt Injection · LLM02 Sensitive Info · LLM06 Excessive Agency).
- 보안/거버넌스: NIST AI RMF 1.0 (2023); Greshake et al. 2023 (indirect prompt injection, AISec).
- 최근: Anthropic 2025 (Agent Skills; memory tool). 일부 사양 · 수치는 공식 문서 확인 권장.